

# Doosri Raay: First Commit submission form answers

WeMakeDevs x AWS Builder Center, First Commit (17-20 Sep 2026). Tracks: Ship It and Build It. Prepared 20 Sep 2026. Text in <ANGLE BRACKETS> is a placeholder to fill before pasting. Repository: <https://github.com/abhay-codes07/doosri-raay> | Live: <https://main.d2inambt66a14p.amplifyapp.com/try>

## Team

| Field                                            | Answer                                                                                                                                                                                                    |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Team leader's WeMakeDevs username *              | <ABHAY_WEMAKEDEVES_USERNAME> (copy it from <a href="https://wemakedevs.org/home">wemakedevs.org/home</a> )                                                                                                |
| Second team member's WeMakeDevs username         | <RAJAT_WEMAKEDEVES_USERNAME>                                                                                                                                                                              |
| Third / fourth team member's WeMakeDevs username | leave blank (two-person team)                                                                                                                                                                             |
| Team leader's GitHub *                           | <a href="https://github.com/abhay-codes07">https://github.com/abhay-codes07</a>                                                                                                                           |
| Second team member's GitHub                      | <a href="https://github.com/rajatnagda45">https://github.com/rajatnagda45</a>                                                                                                                             |
| Team leader's LinkedIn *                         | < <a href="https://www.linkedin.com/in/ABHAY">https://www.linkedin.com/in/ABHAY</a> >                                                                                                                     |
| Second team member's LinkedIn                    | < <a href="https://www.linkedin.com/in/RAJAT">https://www.linkedin.com/in/RAJAT</a> >                                                                                                                     |
| Team leader's resume (public Google Drive link)  | < <a href="https://drive.google.com/file/d/.../view?usp=sharing">https://drive.google.com/file/d/.../view?usp=sharing</a> > Sharing: 'Anyone with the link'. Needed for the Amazon Fast Track interviews. |
| Second team member's resume                      | < <a href="https://drive.google.com/file/d/.../view?usp=sharing">https://drive.google.com/file/d/.../view?usp=sharing</a> >                                                                               |

## Project

| Field                          | Answer                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project title *                | Doosri Raay ("second opinion"): the outsider agent against digital-arrest scams                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Track you are submitting for * | Both. Ship It (the stack is deployed on AWS and live) and Build It (the same code uses the AWS open-source stack: Strands Agents SDK, AWS SAM, LocalStack and Cedar). If the form allows one choice, choose Ship It.                                                                                                                                                                                                                                                                                |
| GitHub link to project *       | <a href="https://github.com/abhay-codes07/doosri-raay">https://github.com/abhay-codes07/doosri-raay</a><br><br>Public, MIT-licensed, 100+ commits between 18 and 20 September with a judge-facing README, docs/ (architecture, API and state-machine contracts, prior art, AI-tools disclosure, demo shot list, research), 230 backend tests and a 70-item evaluation set.                                                                                                                          |
| Deployed link to project       | <a href="https://main.d2inambt66a14p.amplifyapp.com/try">https://main.d2inambt66a14p.amplifyapp.com/try</a><br><br>No sign-up needed. /try signs you into a seeded judge circle: the parent's tile on the left, the guardian's dashboard on the right, a judge FAQ with live timers and a Reset button. Three judge circles exist (judge1, judge2, judge3 @demo.doosriraay.in) so judges never interfere with each other. Password for the optional full sign-in path at /signin: <JUDGE_PASSWORD>. |

| Field                     | Answer                                                                                                                                                                                                                                                                             |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| YouTube video demo link * | <YOUTUBE_URL> (unlisted, 3:00). 0:00 the problem and the idea; 0:18 the parent tile; 0:33 Watch to Ladder in Step Functions; 1:03 the checker; 1:15 Puchho with Polly; 1:27 the recovery case; 2:05 every AWS service in the console; 2:29 sources verified; 2:49 what we learned. |

## What does your project do? (required)

### What problem does your project solve, and who is it for? \*

#### The problem

A "digital arrest" is a scam in which a caller claiming to be the CBI, the police, customs or TRAI keeps a person on a video call for hours or days, orders them to tell no one, and walks them through breaking fixed deposits and wiring the money "for verification". Indians lost Rs 22,495 crore to cyber fraud in 2025 (MHA). Senior citizens alone filed 1,03,488 complaints worth Rs 4,005 crore (MHA reply in the Rajya Sabha, 5 August 2026). Digital arrest accounts for 2,97,727 complaints and Rs 4,057.7 crore since 2022, and in Karnataka the recovery rate for that money fell from 5% in 2025 to 2.2% in early 2026.

We read 25 documented Indian cases from 2025 and 2026 before writing a line of code. In every case that was saved, the save came from an outsider: a bank manager in Pune and Nalgonda, a daughter in Bhopal who showed the fake court order to a lawyer, police reached by a 112 call in Rajkot, or a phone that simply died in Moradabad. Victims who acted alone did so only after reading a newspaper, one of them 165 days in.

Behavioural research says why: 82% of scam losses are payments the victim authorises, warnings habituate by the second exposure, and a person under authority pressure told "questioning felt more dangerous than obeying" will not tap a red button.

Every scam app we found starts from the victim's phone. It listens, shows a warning and expects the victim to act. Doosri Raay starts from the outside.

#### What it does

- Passive isolation ladder (the hero). The parent's app is a genuinely useful Panchang tile: tithi, weather, medicine reminders, a family photo. Opening it each morning is the check-in. If it is not opened by the deadline and a guardian's call goes unanswered, a Step Functions ladder escalates: second guardian, then a named neighbour with a script and the address, then 112 guidance. The parent does nothing, nothing appears on the parent's screen, and nothing is recorded. A scammer watching the phone sees a calendar.
- Recovery case manager. After a loss, the guardian uploads the UPI or bank screenshots. Bedrock extracts the references, code validates them (12-digit UPI/IMPS or 16 to 22 character NEFT/RTGS), the guardian confirms each field beside the image, and the app produces the 1930 call script, an NCRP-compliant narrative (200+ characters, allowed character set, checked in code against the confirmed rows so no hallucinated reference can reach a complaint), the state's e-Zero FIR threshold and the Money Restoration Module checklist. Every legal fact carries its source, date and a "confirm with 1930" caveat. Every human step is a Step Functions callback with a deadline that escalates to a guardian.
- Puchho ("ask"). Two calm buttons on the tile. "Someone says they are police / CBI / bank" plays the official I4C line in Hindi through Amazon Polly and tells the guardians. "Someone says my son is in trouble" pings the son with a family code-word challenge, so a cloned voice is answered by a fact the scammer cannot know.
- Covert SOS. Three taps on the date send location to the guardians with no visible change on screen.
- Screenshot check (a minor tool). One Bedrock call with a forced tool schema returns one of three states, the tactic used (authority, urgency, secrecy, payment switch, verification account) and what to say, in Hindi and English. It never says "safe".

#### Who it is for

The adult son or daughter who lives away from a parent, the parent who wants no lecture and no scary app, and the family circle (a second guardian, a son, a neighbour) that the scam is designed to cut off. Guardians are notification-only: they never see balances, credentials or accounts.

## What is different

Compared with Truecaller Family, Google's fake-call verification, PhonePe Protect and hackathon projects such as Kavach, Rakshak and SwarVed: zero action required from the victim; no warning on the parent's screen at all; a validated end-to-end recovery pipeline through NCRP, e-Zero FIR and the refund module; hashed, re-verifiable sources behind every legal fact; and a deployed product with an evaluation harness and a test set.

## How did you use AWS in your project? (required)

---

**Build It: the AWS open-source stack. Ship It: the AWS services. \***

### Ship It: the AWS services

One AWS SAM stack in `ap-south-1` (`infra/template.yaml`, 28 resources) plus Amplify Hosting. Nine services do product work; five keep it running.

- Amazon Bedrock. Claude Sonnet 4.6 (`global.anthropic.claude-sonnet-4-6`) with Claude Haiku 4.5 as the fallback, reached from Mumbai through global cross-region inference profiles via the Converse API. The checker makes one Converse call with forced tool use (`toolChoice` pinned to a tool schema), so the answer is always an enum of three states, an enum of tactics and capped strings; the screenshot or message is passed as untrusted content. The recovery agent uses Bedrock for vision extraction into a tool schema and for the NCRP narrative, which code re-checks against the confirmed database rows.
- AWS Step Functions (Standard). Three state machines. Watch is started by every check-in with the next deadline in its input; if no newer check-in arrives it starts Ladder, so the trigger is itself an execution in the console and there is no scheduler. Ladder and RecoveryCase model every human step as `lambda:invoke.waitForTaskToken` with `TimeoutSecondsPath` read from the execution input, so one definition runs 15-minute rungs in production and 45-second rungs on camera.
- AWS Lambda (Python 3.12). Six functions from one code directory: `api`, `classify-worker` (invoked asynchronously so `POST /analyze` returns 202 and the client polls), `ladder-task` (writes the task with its token and sends Web Push), `watch-check`, `ladder-status`, and `recovery-agent` as a container image for the Strands SDK. X-Ray tracing is on, with the X-Ray SDK patching `boto3`.
- Amazon API Gateway (HTTP API). Every route behind a Cognito JWT authorizer; the caller's circle is derived from their own profile item, never from the request; stage throttling and a per-user daily quota as cost guards.
- Amazon Cognito. One user pool and app client; the frontend sends the ID token, which the authorizer validates against the client id; the `/try` and `/demo` pages hold a second, parent identity in memory only.
- Amazon DynamoDB. One single-table design for profiles, circle members, check-ins, tasks (carrying the Step Functions task token and a TTL), reports, cases and quotas, with a GSI for id lookups and point-in-time recovery.
- Amazon S3. One private bucket: presigned POST uploads with `content-length-range` (3.5 MB) and a `Content-Type` condition, `SSE-S3`, Block Public Access, a 7-day lifecycle on screenshots, the cached Polly clip, and the hashed official sources.
- Amazon Polly. The neural Hindi voice (Kajal, Aditi fallback) renders the I4C advisory line for the Puchho button; synthesised once, cached in S3.
- AWS Amplify Hosting. Serves the Vite PWA from `frontend/` with `amplify.yml` and an SPA rewrite rule; `VITE_*` variables come from the stack outputs.
- Operations: Systems Manager Parameter Store (a `SecureString` for the Web Push private key), Amazon ECR (the `recovery-agent` image pushed by `sam deploy`), AWS Budgets (USD 20 and USD 50 alarms), CloudWatch Logs (14-day groups per function and state machine, ERROR level with execution data off so tokens are never logged) and AWS X-Ray.

### Build It: the AWS open-source stack

- Strands Agents SDK runs the recovery agent's tool loop. The tools (`extract_transactions`, `validate_fields`, `lookup_ezero_threshold`, `mrm_eligibility`, `draft_narrative`) are plain Python functions, capped at 12 tool calls,

and a deterministic path runs the same functions in order if the agent is unavailable or fails, so the pipeline never depends on the agent being healthy.

- AWS SAM CLI builds, validates and deploys the whole stack (sam validate --lint, sam build --use-container, sam deploy) and runs the API on a laptop with sam local start-api.
- LocalStack serves DynamoDB and S3 locally (docker-compose.yml; make local-up, local-bootstrap, local-api). The backend honours AWS\_ENDPOINT\_URL and stubs Step Functions in local mode. We say plainly which routes work locally and which need the cloud.
- Cedar is the authorization layer. backend/common/authz/policies.cedar decides which role may list which task kinds, complete a task, read a case or reset the demo. Forbid rules carry @id annotations that become bilingual 403 messages ("covert tasks are hidden from the parent", "guardians are notification-only"), and a 16-row policy table is tested against the Rust engine (cedarpy) and a fallback evaluator of the same file.

### The organisers' other techniques, applied

Rules as data: every legal fact (e-Zero FIR thresholds by state, Money Restoration Module rules, NCRP form rules) lives in a JSON file with operator, value, source, quoted sentence and caveat, and a test fails if a threshold or an outlet name appears in Python. Documents you can prove: scripts/verify\_sources.py downloads every cited government source, hashes it with SHA-256 and checks every quoted sentence against the live page (12 of 13 sources fetched and all 25 quotes found on 20 September 2026); the app shows the same hashes beside the facts in a Sources panel. A constrained, checked model: three states, never "safe", forced tool use, enum checks in code, untrusted-input wrapping, and the narrative re-check. When Bedrock is unavailable, the checker falls back to a rule-based check and the card says so; the recovery case falls back to manual transaction entry.

## Blog links

---

### Up to four Builder Center blog links, one per member

<BUILDER\_CENTER\_BLOG\_URL\_ABHAY> Draft ready in docs/BLOG.md: "We built the outsider: why our scam project has no scam warning on the parent's screen".

<BUILDER\_CENTER\_BLOG\_URL\_RAJAT> Optional second post, for example "Deploying a Strands + Step Functions stack with SAM in one evening".

## Contributions

---

### Team leader's contributions \*

Abhay Singh (team leader): research, architecture, backend, infrastructure.

- Research and the pivot: the 25-case study of 2025-26 digital-arrest incidents and the review of intervention evidence that moved the design from a victim-side detector to the outsider ladder; the sourced problem numbers; the prior-art review.
- Architecture and contracts: the API, data-model and state-machine contracts, the single-table DynamoDB design, the self-renewing Watch that replaces a scheduler.
- Backend: the API handlers, the Bedrock checker with forced tool use and untrusted-input wrapping, the Strands recovery agent with plain-code tools and a deterministic fallback, the NCRP narrative re-check, rules as data, the sources verification script and GET /sources, the Cedar authorization layer, the Polly clip cache, Web Push, quotas, the 230-test suite, the 70-item Hindi/English evaluation set and harness.
- Infrastructure: the SAM template (Cognito, HTTP API with JWT authorizer, six Lambdas including the container image, hardened S3, DynamoDB, three Step Functions definitions with waitForTaskToken and TimeoutSecondsPath, log groups, Budgets, SSM), the Makefile, LocalStack local mode, the first-deploy checklist.
- Documentation: README, submission text, demo shot list, AI-tools disclosure.

<Adjust to what you actually did; keep it truthful to the commit history.>

## Second team member's contributions

Rajat Nagda: frontend, deployment and operations, demo.

- Frontend: the Vite + React PWA with a Hindi-first custom auth flow, the parent's Panchang tile with passive check-in, covert SOS and Puchho, the guardian dashboard with task cards and the ladder status, case intake and case detail with source lines and live timers, Settings (family photo, holiday mode), the /demo split view and the no-sign-up /try judge route with its FAQ, the service worker and Web Push, amplify.yml.
- Deployment and operations: the AWS account and Bedrock model access, sam build and deploy, the SSM key, Amplify Hosting with the SPA rewrite rule, seeding the demo and the three judge circles, uploading the hashed sources to S3, the live smoke test, the eval run against Bedrock.
- Demo: the 3-minute video, the Hindi copy review, the blog post.

<Adjust to what Rajat actually did.>

## Help us evaluate you: your feedback on the AWS services you used (required)

---

### What you did not like, and what could be better \*

- Bedrock from Mumbai means global inference profiles. Claude Sonnet 4.6 and Haiku 4.5 are reachable from ap-south-1 only through global.anthropic.\* profile ids, and the IAM policy must cover the profile and the underlying model in every region it can route to. Our stack is in ap-south-1; inference is not. The model page should say this up front. Amazon Nova 2 Sonic, which we wanted for the Hindi voice line, is not available in ap-south-1 and needs a bidirectional stream rather than Lambda, so Puchho uses Polly.
- Bedrock is not in the Free tier, the Free plan has near-zero Bedrock quota, and a brand-new account sits on a verification hold before the first model call succeeds. The error you get until then does not look like an account problem. A banner in the Bedrock console, and a way to request the check early, would save every student team an evening.
- Strands does not fit a zip Lambda comfortably. With boto3 and Pillow the bundle grew until we moved the agent to a container image, which means ECR, Docker on every laptop, --platform linux/amd64 and BUILDX\_NO\_DEFAULT\_ATTESTATIONS=1, because Lambda rejects the multi-manifest OCI index that BuildKit now produces by default. None of this is in the Strands Lambda guide. Strands also has no maximum-iterations option, so we added our own tool-call budget.
- sam deploy --template infra/template.yaml is a trap. Passing the source template makes SAM skip the built template, so a PackageType: Image function has no ImageUri and the deploy fails with an unhelpful message. SAM could warn when an image function is deployed from an unbuilt template.
- The HTTP API JWT authorizer wants the Cognito ID token. The access token has no aud claim, so every route returns 401 until you switch, and neither the Amplify fetchAuthSession docs nor the authorizer docs say which token to send.
- sam local start-api does not emulate the JWT authorizer, so local requests carry no claims and we added a dev-only identity fallback. LocalStack community edition has no Step Functions, Bedrock or Polly, so "the whole stack locally" is honest only for the API, DynamoDB and S3. A community-tier Step Functions emulator would complete the story.
- Amazon SES stays sandboxed for a hackathon and Amazon SNS SMS to Indian numbers needs DLT registration. Both are reasonable, but the consoles could say so before you spend an evening on it; we dropped e-mail and SMS for in-app tasks and Web Push.
- Amplify Hosting needs an SPA rewrite rule for deep links, and the rule cannot be expressed in amplify.yml; the Vite preset should add it by default. amplify.yml also does not pin Node, so a Vite 7 build can fail on the image's default runtime.
- waitForTaskToken with TimeoutSecondsPath is exactly the pattern for "a human has N minutes before we escalate" with a per-environment N, and we found it in the ASL reference rather than in a tutorial or a Workflow Studio template. A timeout also does not close the task record on the application side, so stale tasks

must be superseded by the application.

- X-Ray with Tracing: Active alone shows one node. The DynamoDB, Bedrock and Step Functions segments need the X-Ray SDK patching in code, and the empty trace map does not say so.
- Polly has no per-resource console object and CloudFormation cannot create a SecureString SSM parameter, so two of our services can only be shown in a video through an IAM statement and a placeholder overwrite.
- Cedar's semantics are excellent (forbid beats permit, and the decision names the policy), but the Python binding names policies policy0..n, so @id annotations must be mapped by file order. First-class annotation lookup in the bindings would help.

## What did you like about the AWS services you used? (required)

---

### What worked well \*

- Step Functions is the product, not the plumbing. Watching the Watch execution expire, StartLadder fire and Ladder sit at Rung1GuardianCall waiting for a human is the whole idea in one screen. Standard workflows with waitForTaskToken and TimeoutSecondsPath let one definition run 15-minute rungs in production and 45-second rungs in the demo with no code change, and the execution graph is the best visual we have.
- Bedrock Converse with forced tool use is structured output done right. One call with toolChoice pinned to a tool schema replaced a JSON parser, a retry loop and most of the prompt; the model can only answer inside our enums. Global cross-region profiles meant Sonnet 4.6 and Haiku 4.5 worked from ap-south-1 the moment access was granted, and swapping the fallback model is one environment variable.
- AWS SAM made a 28-resource stack a single template: Cognito, an HTTP API with a JWT authorizer, six Lambdas including a container image, DynamoDB, S3, three state machines, log groups, Budgets and an SSM parameter. Policy templates such as DynamoDBCrudPolicy kept IAM short, sam validate --lint caught mistakes before any deploy, and DefinitionSubstitutions let the ASL files stay readable JSON with real ARNs injected at deploy.
- The HTTP API JWT authorizer plus Cognito gave per-route authentication with zero code: the caller's sub arrives in the request context and our handlers derive everything else from it, so no request can name a circle it does not belong to.
- DynamoDB single-table design with TTL fit the domain: tasks carry their Step Functions token and expiry, reports and SOS items clean themselves up, and on-demand capacity means the demo stack costs nothing while idle.
- S3 presigned POST with a policy (content-length-range, Content-Type starts-with) was the cleanest way to let a browser upload straight to a private bucket with a hard size cap and no proxying through Lambda.
- Amazon Polly's neural Hindi voice reads the I4C advisory line naturally, and synthesising once and caching the MP3 in S3 made the feature effectively free.
- Amplify Hosting connected to the GitHub repo with amplify.yml gave us a build on every push and a URL for the form; environment variables from the stack outputs made frontend configuration a one-command step.
- The AWS open-source stack the Build It track pointed us to was genuinely useful. Strands let the tools be plain Python functions we could unit-test without the agent, and the model provider is swappable. LocalStack plus sam local gave a laptop loop for the API, DynamoDB and S3. Cedar's forbid-beats-permit semantics let us express "guardians are notification-only" and "the parent never sees covert tasks" as five policies instead of scattered conditionals, with the denial id surfacing as a bilingual message.
- Onboarding: the Builder Center student credits covered the whole weekend, and the Bedrock playground earning an extra credit was a good nudge to test model access before writing code.

## Before pasting

---

### Checklist

- The repository is public.
- Fill every <PLACEHOLDER>: WeMakeDevs usernames, LinkedIn, resumes, YouTube link, judge password, blog links. The live URL is already filled.
- Read the two contribution answers and correct them to what each of you actually did.
- The form can be edited until the deadline (the countdown on the submit page is the authoritative time), so submit early and refine.
- Both team members must be student-verified on AWS Builder Center or the entry is not scored.